Fachklausur in POS (Java)

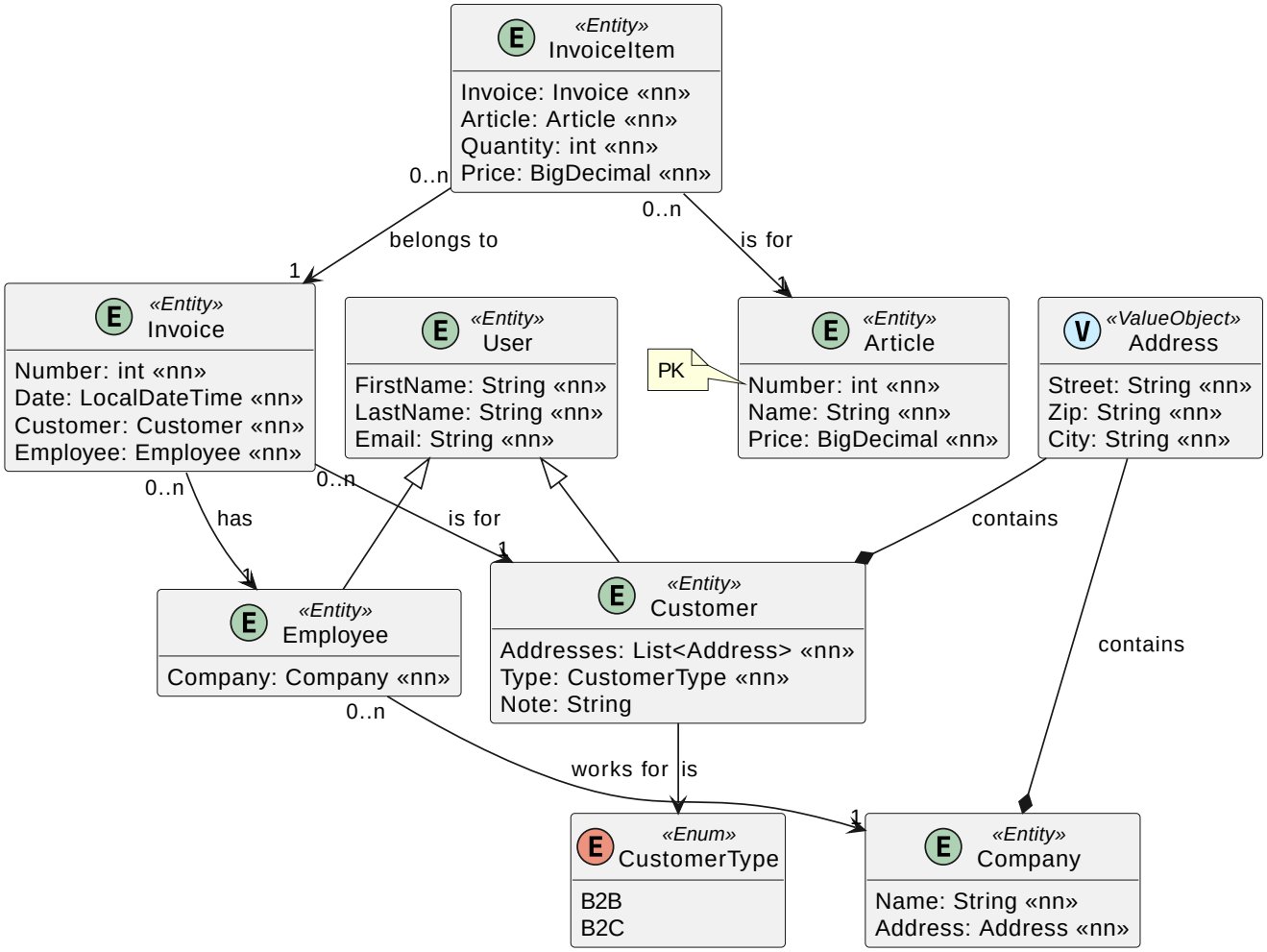
> Klassen: 6AAIF, 6BAIF, 6CAIF, 6AKIF, 6BKIF
> Datum: MI, 4. Juni 2025
> Arbeitszeit: 5 UE

Auf Ihrem Laufwerk R befindet sich der Ordner *SPG_Fachtheorie*. Darin befindet sich ein Gradle Multi-Module Projekt mit den Modulen *aufgabe1*, *aufgabe2* und *aufgabe3*. Öffnen Sie den Ordner in IntelliJ IDEA. Arbeiten Sie ausschließlich in diesem Projekt. Da das Projekt auf dem Netzlaufwerk liegt, muss am Ende nicht extra auf ein Abgabelaufwerk kopiert werden. Beenden Sie am Ende Ihrer Arbeit alle Programme und lassen Sie den PC eingeschaltet.

> **!** Füllen Sie die Datei *README.md* in *SPG_Fachtheorie/README.md* mit Ihren Daten (Klasse, Name und Accountname) aus. **Falls Sie dies nicht machen, kann Ihre Arbeit nicht bewertet werden!**

Teilaufgabe 1: Object Relation Mapping

Ein kleines Kundenverwaltungssystem soll mit Hilfe von JPA/Hibernate und Spring Boot implementiert werden. Es soll Kunden (*Customer*) und Artikel (*Article*) erfassen können. Wenn Kunden einen oder mehrere Artikel kaufen, wird eine Rechnung (*Invoice*) angelegt. Die Rechnung hat mehrere Zeilen (*InvoiceItem*). Die Zeile beschreibt, in welcher Menge und zu welchem Preis ein konkreter Artikel gekauft wurde. Beachten Sie dabei die geforderten Anforderungen an einzelne Modellklassen.



Arbeitsauftrag

Erstellung der Modelklassen

Implementieren Sie das dargestellte Diagramm als JPA Entity-Klassen. Im Modul *aufgabe1* befinden sich leere Klassen sowie die Klasse *BaseEntity*, die Sie nutzen sollen. Beachten Sie bei der Umsetzung folgende Punkte:

- Legen Sie nötige Konstruktoren an. Ein *public* Konstruktor soll alle im Modell enthaltenen Felder initialisieren. Ergänzen Sie die für JPA nötigen *protected* No-Arg Konstruktoren.
- Beachten Sie Attribute Constraints wie **not null** (`nn`).
- Strings sollen - wenn nicht anders angegeben - mit einer Maximallänge von 255 Zeichen definiert werden (JPA Default).
- Der Preis soll mit 9 Stellen (5 Vorkomma- und 4 Nachkommastellen) gespeichert werden. Sie können `@Column(precision = 9, scale = 4)` verwenden.
- Das Entity *Address* soll in *Company* und *Customer* als *Value Object* definiert werden. Beachten Sie, dass in *Customer* eine Liste von Value Objects definiert werden muss.
- Verwenden Sie eigene primary keys mit dem Namen *id* (autoincrement über *BaseEntity*), außer im Modell ist mit *PK* explizit ein Schlüssel angegeben.
- Speichern Sie die enum *CustomerType* in *Customer* als String in der Datenbank ab.

- Die Klassen *Employee* und *Customer* erben von der Klasse *User*. Verwenden Sie Single Table Inheritance.
- Erstellen Sie passende Spring Data JPA Repository-Interfaces im Package *persistence*.

Damit Sie eine Liste von Value Objects mit JPA erstellen können, verwenden Sie folgende Konfiguration:

```
@ElementCollection
private List<Address> addresses = new ArrayList<>();
```

Verfassen von Tests

In der Klasse *InvoiceContextTests* im Modul *aufgabe1* (unter *test*) sollen Testmethoden verfasst werden, die die Richtigkeit Ihrer Implementierung beweisen sollen.

- *persistEnumSuccessTest* beweist, dass die Enum *CustomerType* in *Customer* korrekt gespeichert wird.
- *persistValueObjectInCompanySuccessTest* beweist, dass Sie ein Entity vom Typ *Company* mit einer Adresse als Value Object speichern können.
- *persistValueObjectsInCustomerSuccessTest* beweist, dass Sie ein Entity vom Typ *Customer* mit einer Liste von Adressen als Value Object speichern können.
- *persistInvoiceItemSuccessTest* beweist, dass Sie ein Entity vom Typ *InvoiceItem* in der Datenbank speichern können.

Bewertung

Aufgabe 1 (33 Punkte in Summe, 36 %)	**Punkte**
Article: Alle geforderten Constraints und Beschränkungen wurden umgesetzt.	1
Article: Die Klasse verwendet Number als primary key.	1
Article: Der vorgegebene Test InsertArticleTest läuft durch.	2
Address: Die Klasse enthält alle in der Angabe vorgesehenen Properties.	1
Address: Die Klasse besitzt keine ID und wird nicht als Tabelle verwendet.	1
Company: Alle geforderten Constraints und Beschränkungen wurden umgesetzt.	1
Company: Die Klasse besitzt ein korrekt konfiguriertes Value Object "Address".	1
Company: Der vorgegebene Test InsertCompanyTest läuft durch.	2
Employee: Alle geforderten Constraints und Beschränkungen wurden umgesetzt.	1
Employee: Der vorgegebene Test InsertEmployeeTest läuft durch.	2
Customer: Alle geforderten Constraints und Beschränkungen wurden umgesetzt.	1
Customer: Die Enumeration Type wird als String in der Datenbank gespeichert.	1

Aufgabe 1 (33 Punkte in Summe, 36 %)	Punkte
Customer: Der vorgegebene Test InsertCustomerTest läuft durch.	2
Customer: Der vorgegebene Test InsertCustomerWithAddressesTest läuft durch.	2
Invoice: Alle geforderten Constraints und Beschränkungen wurden umgesetzt.	1
Invoice: Der vorgegebene Test InsertInvoiceTest läuft durch.	2
InvoiceItem: Alle geforderten Constraints und Beschränkungen wurden umgesetzt.	1
InvoiceItem: Der vorgegebene Test InsertInvoiceItemTest läuft durch.	2
persistEnumSuccessTest: Der Test ist richtig aufgebaut.	1
persistEnumSuccessTest: Der Test läuft durch.	1
persistValueObjectInCompanySuccessTest: Der Test ist richtig aufgebaut.	1
persistValueObjectInCompanySuccessTest: Der Test läuft durch.	1
persistValueObjectInCustomerSuccessTest: Der Test ist richtig aufgebaut.	1
persistValueObjectInCustomerSuccessTest: Der Test läuft durch.	1
persistInvoiceItemSuccessTest: Der Test ist richtig aufgebaut.	1
persistInvoiceItemSuccessTest: Der Test läuft durch.	1

Teilaufgabe 2: Abfragen und Servicemethoden

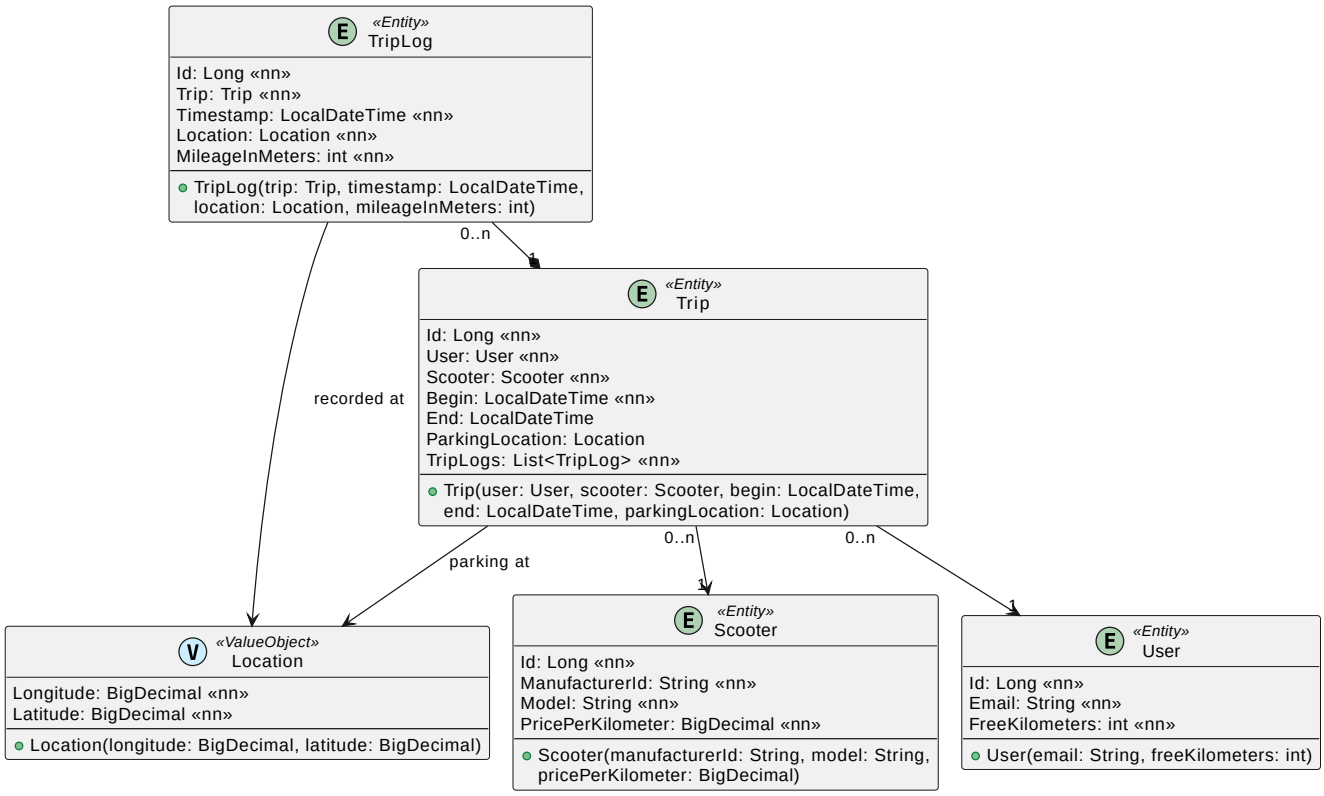
Das nachfolgende Modell zeigt eine Umsetzung eines Scooter Sharing Services. Kunden können zwischen 2 Abrechnungsmethoden wählen:

- *Pay as you go:* Der Kunde zahlt pro gefahrenen Kilometer.
- *Included Kilometers:* Der Kunde zahlt einen monatlichen Beitrag und erhält eine bestimmte Anzahl an Freikilometern. Die freien Kilometer gelten pro Fahrt. Hat ein Kunde z. B. 2 Freikilometer, so wird bei Fahrt A, die 3 km lang ist, 1 km berechnet. Bei Fahrt B, die 4 km lang ist, werden 2 km berechnet.

Beim Entsperren mit der App wird ein *Trip* gestartet. Der integrierte Tracker des Scooters übermittelt in regelmäßigen Abständen die Position und den Kilometerstand als *TripLog*. Dadurch ist eine Abrechnung auch dann möglich, wenn kein durchgehendes GPS Signal verfügbar ist (z. B. Unterführungen, etc). Die Länge einer Fahrt (*Trip*) ist dann die Differenz zwischen niedrigsten und höchsten Kilometerstand der *TripLogs* dieser Fahrt.

Die Preise für den gefahrenen Kilometer variieren je nach Modell und sind in der *Scooter* Klasse hinterlegt. Scooter mit Sitz und hoher Reichweite haben z. B. einen höheren Preis pro Kilometer als Scooter ohne Sitz.

Wird ein Scooter abgestellt, wird der *Trip* beendet und die Felder *End* und *ParkingLocation* werden gesetzt. So ist erkennbar, welche Trips noch laufen und welche bereits beendet sind.



Arbeitsauftrag

Im Modul *aufgabe2* stehen vollständig implementierte JPA Entities und Spring Data JPA Repositories zur Verfügung. Das Model ist bereits im Package *model* implementiert. **Sie müssen keine Implementierung des Modelles vornehmen, sondern es in Ihren Servicemethoden verwenden.**

Implementierung von Servicemethoden

Führen Sie Ihre Implementierungen in *ScooterService.java* im Package *service* durch. Verwenden Sie die über Dependency Injection bereitgestellten Repositories.

public Trip addTrip(Long userId, Long scooterId, LocalDateTime begin)

Diese Methode soll einen Trip in der Datenbank anlegen. Dabei sollen folgende Randbedingungen geprüft werden:

- Wird die *userId* nicht gefunden, so ist eine *ScooterServiceException* mit dem Text *Invalid user.* zu werfen.
- Wird die *scooterId* nicht gefunden, so ist eine *ScooterServiceException* mit dem Text *Invalid scooter.* zu werfen.
- Existiert ein offener Trip für diesen Scooter in der Datenbank, so ist eine *ScooterServiceException* mit dem Text *Scooter is not parked.* zu werfen. Sie erkennen offene Trips daran, dass *Trip.end* keinen Wert (*null*) hat.

Sind alle Bedingungen erfüllt, fügen Sie den neuen Trip mit dem übergebenen User, Scooter und der Beginnzeit in die Datenbank ein.

Verwenden Sie den bereitgestellten Unittest addTripTest in ScooterServiceTests.java, um die Richtig-

keit Ihrer Methode zu prüfen.

public List<TripDto> getTripInfos(LocalDateTime beginFrom, LocalDateTime beginTo)

Diese Methode soll Informationen zu allen Trips abfragen, die in einem gewissen Zeitraum begonnen haben. In Ihrer Serviceklasse ist ein Record als Rückgabetyp definiert:

```
public record TripDto(
    Long id, LocalDateTime begin, LocalDateTime end, Long scooterId,
    String scooterModel, Long userId, String userEmail, boolean isParked) {}
```

Das Property *isParked* hat den Wert *true* wenn *Trip.end* nicht null ist. Fragen Sie die benötigten Informationen zur Erstellung von *TripDto* aus der Datenbank ab.

Verwenden Sie den bereitgestellten Unittest getTripInfosTest in ScooterServiceTests.java, um die Richtigkeit Ihrer Methode zu prüfen.

public int calculateTripLength(Long tripId)

Diese Methode soll die Länge eines Trips in Metern berechnen. Ein Trip hat mehrere *TripLogs*. Dies sind Punkte, die z. B. von einem GPS Modul am Scooter übermittelt werden. Um die Länge eines Trips in Metern zu berechnen, verwenden Sie den kleinsten und größten Kilometerstand (*TripLog.mileageInMeters*) des Logs dieses Trips. Die Differenz zwischen diesen beiden Werten ist der gesuchte Wert.

Verwenden Sie den bereitgestellten Unittest calculateTripLengthTest in ScooterServiceTests.java, um die Richtigkeit Ihrer Methode zu prüfen.

public BigDecimal calculatePrice(Long tripId)

Diese Methode soll den Preis für einen Trip berechnen. Dabei müssen folgende Werte berücksichtigt werden:

- Ein User hat freie Kilometer (*User.freeKilometers*). Diese müssen von der Triplänge abgezogen werden.
- Der Preis pro Kilometer ist in *Scooter.pricePerKilometer* gespeichert.

Wird der Trip nicht gefunden, so ist der Preis 0 zurückzugeben.

Verwenden Sie den bereitgestellten Unittest calculatePriceTest in ScooterServiceTests.java, um die Richtigkeit Ihrer Methode zu prüfen.

Bewertung

Aufgabe 2 (26 Punkte in Summe, 28 %)	Punkte
addTrip: Die Methode berücksichtigt alle Randbedingungen korrekt.	2
addTrip: Die Methode fügt den neuen Trip korrekt in die Datenbank ein.	1

Aufgabe 2 (26 Punkte in Summe, 28 %)	Punkte
addTrip: Der Unittest addTripTest läuft durch.	3
getTripInfos: Die Methode filtert korrekt die Daten.	1
getTripInfos: Die Methode fragt korrekt ab und projiziert das Ergebnis auf die DTO Klasse.	2
getTripInfos: Der Unittest getTripInfosTest läuft durch.	3
calculateTripLength: Die Methode filtert korrekt die Daten.	1
calculateTripLength: Die Methode führt die Berechnung korrekt durch.	3
calculateTripLength: Der Unittest calculateTripLengthTest läuft durch.	3
calculatePrice: Die Methode filtert korrekt die Daten.	1
calculatePrice: Die Methode führt die Berechnung korrekt durch.	3
calculatePrice: Der Unittest calculatePriceTest läuft durch.	3

Teilaufgabe 3: REST(ful) API

Für das vorige Modell der Scooter Sharing App soll eine RESTful API implementiert werden. Wenn Sie das Modul *aufgabe2* starten, steht Ihnen unter der URL *http://localhost:8080/swagger-ui.html* ein Endpoint Explorer zur Verfügung. Die Datenbank beinhaltet Musterdaten (via *data.sql*), sodass Sie die Funktionalität Ihrer Controller damit testen können.

Arbeitsauftrag

Implementieren Sie die folgenden REST API Routen. In *TripsController.java* im Package *aufgabe3.controller* steht dafür ein leerer Controller bereit. Über Dependency Injection wird ein mit Musterdaten gefülltes *TripRepository* bereitgestellt. Sie können die Abfragen direkt in den Controllermethoden implementieren. Achten Sie bei Fehlern auf eine RFC-9457 kompatible Antwort, indem Sie `ProblemDetail.forStatusAndDetail()` mit geeignetem Statuscode verwenden.

GET /trips/{id}?includeLog=true

Diese REST API Route soll einen bestimmten Trip mitsamt der Logeinträge retournieren. Wird der optionale Parameter *includeLog* mit dem Wert *true* mitgegeben (*false* ist der Default-Wert), soll die Antwort die Logeinträge enthalten. Ist der Parameter *false*, soll ein leeres Array zurückgegeben werden.

Erstellen Sie zuerst die Felder in den Klassen *TripResponse.java* und *TripLogResponse.java* im Package *aufgabe3.viewmodels*, sodass die Daten dem nachfolgenden Schema entsprechen.

Table 1. Erwartete HTTP-Antworten:

HTTP Status	Bedingung
200	Ein *TripResponse* mit leerem Array für *logs*, wenn der Parameter *includeLog* false ist.

HTTP Status	Bedingung
200	Ein *TripResponse* mit einem Array von *TripLogResponse*, wenn der Parameter *includeLog* true ist.
200	Ein *TripResponse*, welches optional *TripLogResponses* enthält.
404	Für eine unbekannte Id. (inkl. RFC-9457 ProblemDetail im Body)

Response für GET /trips/1?includeLog=false

```
{
  "id": 1,
  "userEmail": "alice@example.com",
  "scooterManufacturer": "Xiaomi",
  "begin": "2025-02-07T10:46:03",
  "end": null,
  "logs": []
}
```

Response für GET /trips/1?includeLog=true

```
{
  "id": 1,
  "userEmail": "alice@example.com",
  "scooterManufacturer": "Xiaomi",
  "begin": "2025-02-07T10:46:03",
  "end": null,
  "logs": [
    {
      "timestamp": "2025-02-07T10:46:03",
      "longitude": 16.48055,
      "latitude": 48.16812,
      "mileageInMeters": 12397
    },
    {
      "timestamp": "2025-02-07T10:48:03",
      "longitude": 16.60784,
      "latitude": 48.30346,
      "mileageInMeters": 12663
    },
    {
      "timestamp": "2025-02-07T10:52:03",
      "longitude": 15.85092,
      "latitude": 48.21999,
      "mileageInMeters": 13510
    },
    {
      "timestamp": "2025-02-07T10:54:03",
      "longitude": 15.44802,
      "latitude": 48.26622,
      "mileageInMeters": 14396
```

```
    }
  ]
}
```

PATCH /trips/{id}

Dieser Endpunkt soll die Daten am Ende eines Trips aktualisieren. Dabei werden die Felder *end* und *parkingLocation* auf die entsprechenden Werte gesetzt. Achten Sie darauf, dass der Wert von *Trip.end* vor der Aktualisierung *null* sein muss. Es dürfen keine schon beendeten Trips nochmals beendet werden. Ist der Trip bereits beendet, liefern Sie HTTP 400 mit der Meldung *Trip already ended.*

Verwenden Sie das Request Object in *UpdateTripRequest.java* im Package *aufgabe3.viewmodels*, um den Payload zu typisieren. Achten Sie darauf, dass nur gültige Werte für *longitude* (-180 bis +180) und *latitude* (-90 bis +90) übermittelt werden können. Stellen Sie dies durch entsprechende Jakarta Validation Annotations im Request Object sicher.

Der Trip mit der ID 4 ist nicht beendet, er kann zum Testen verwendet werden.

Table 2. Erwartete HTTP-Antworten:

HTTP Status	Bedingung
204	No content, wenn erfolgreich aktualisiert wurde.
400	Wenn ein Trip, der bereits beendet ist, modifiziert werden soll. (inkl. RFC-9457 ProblemDetail im Body)
400	Wenn die Werte für *longitude* oder *latitude* ungültig sind. (inkl. RFC-9457 ProblemDetail im Body)
404	Für eine unbekannte Id. (inkl. RFC-9457 ProblemDetail im Body)

Payload für PATCH /trips/4

```
{
  "end": "2025-05-30T10:13:29",
  "longitude": 16.4,
  "latitude": 48.1
}
```

Verfassen von Integration Tests

In *TripsControllerIT.java* (unter *test*) sollen Integration Tests verfasst werden. Es soll jede Zeile in den Tabellen bei *Erwartete HTTP-Antworten* geprüft werden. Verwenden Sie dafür REST Assured und die per `@Autowired` bereitgestellten Repositories:

Methode / Annotation	Beschreibung
`@SpringBootTest(webEnvironment = RANDOM_PORT)`	Startet die Applikation auf einem zufälligen Port.

Methode / Annotation	Beschreibung
`@LocalServerPort`	Injiziert den Port in eine Variable.
`RestAssured.port = port`	Setzt den Port für REST Assured.
`given().when().get("/trips/{id}", 1).then().statusCode(200)`	Beispiel für einen GET Request.
`given().contentType(ContentType.JSON).body(command).when().patch("/trips/{id}", 4).then().statusCode(204)`	Beispiel für einen PATCH Request.

Bewertung

Aufgabe 3 (33 Punkte in Summe, 36 %)	Punkte
GET /trips/{id} filtert die Daten korrekt.	1
GET /trips/{id} liefert das korrekte Verhalten im Fall einer nicht vorhandenen Id.	1
GET /trips/{id} verwendet den Query Parameter includeLog korrekt.	3
Die Klasse TripResponse wurde korrekt implementiert.	2
Die Klasse TripLogResponse wurde korrekt implementiert.	1
GET /trips/{id} liefert das korrekte Ergebnis im Erfolgsfall für includeLog = false.	2
GET /trips/{id} liefert das korrekte Ergebnis im Erfolgsfall für includeLog = true.	2
Der Integration Test beweist die korrekte Implementierung von GET /trips/{id} im Erfolgsfall.	2
Der Integration Test beweist die korrekte Implementierung von GET /trips/{id} bei Fehlerzuständen.	2
Der Integration Test für GET /trips/{id} läuft durch.	3
PATCH /trips/{id} fragt korrekt die Werte aus der Datenbank ab.	1
PATCH /trips/{id} liefert das korrekte Verhalten im Fall einer nicht vorhandenen Id.	1
PATCH /trips/{id} liefert das korrekte Verhalten im Fall eines bereits beendeten Trips.	1
PATCH /trips/{id} aktualisiert den Wert in der Datenbank.	2
Die Request Klasse UpdateTripRequest kann den Payload typisieren.	1
Die Request Klasse UpdateTripRequest besitzt die definierten Validierungen.	1
Der Integration Test beweist die korrekte Implementierung von PATCH /trips/{id} im Erfolgsfall.	2
Der Integration Test beweist die korrekte Implementierung von PATCH /trips/{id} bei Fehlerzuständen.	2
Der Integration Test für PATCH /trips/{id} läuft durch.	3

Beurteilung:

92 - 81 Punkte: Sehr gut (1)
80 - 70 Punkte: Gut (2)
69 - 58 Punkte: Befriedigend (3)
57 - 46 Punkte: Genügend (4)
unter 46 Punkte: Nicht genügend (5)

Viel Erfolg!